

프로그래밍응용개념

#03 변수, 타입, 연산자

국립한밭대학교 인공지능 소프트웨어학과

이성민

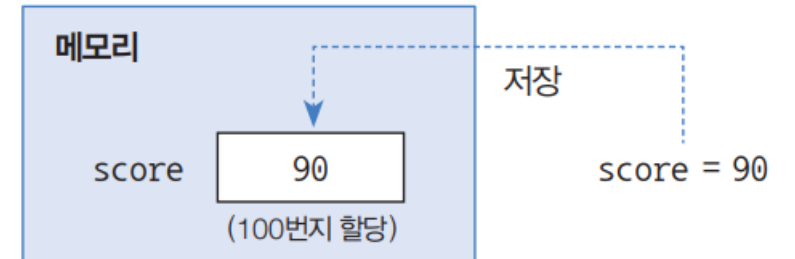
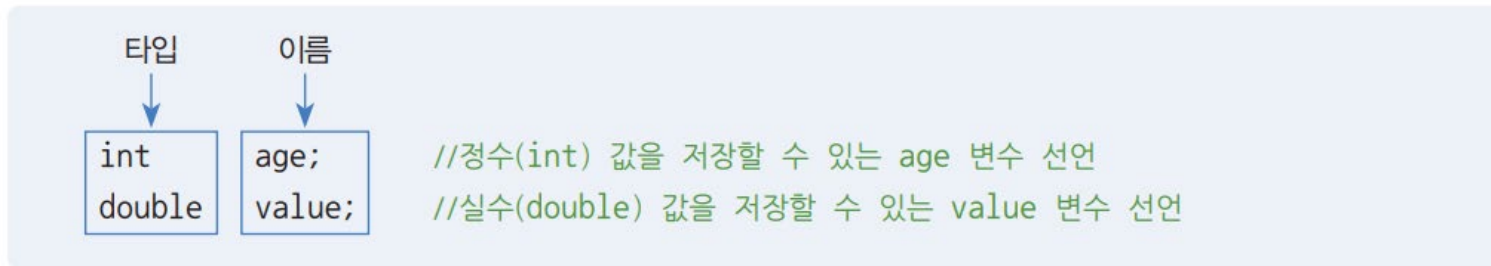
변수

- 정수, 실수, 문자, 불리언 자료형
- 문자열 메소드
- 타입 추론

변수

- 변수 선언

- 변수를 사용하려고 하면 변수 선언이 필수
- 어떤 타입의 데이터를 저장할 것인지, 변수 이름이 무엇인지 결정
- 변수에 최초로 값이 대입될 때 메모리에 할당되고, 해당 메모리에 값 저장



변수

- 변수의 이름
 - Camel 스타일 (여러 단어를 혼합하는 경우 대문자로 띄어쓰기 구분)
 - 변수명은 소문자로 시작
 - score
 - mathScore
 - sportscar
- 소스 파일명(클래스명)은 대문자로 시작
 - Week.java
 - MemberGrade.java
 - ProductKind.java

변수

- 변수 초기화
 - 변수에 최초로 값을 넣는 행위
 - 변수가 사용되기 이전에 초기화 되어 있어야 함

```
//변수 value 선언  
int value;  
  
//연산 결과를 변수 result의 초기값으로 대입  
int result = value + 10;  
  
//변수 result 값을 읽고 콘솔에 출력  
System.out.println(result);
```

정수 타입 (정수 자료형)

- 정수 타입 종류

- 변수 타입에 따라 메모리에 할당되는 크기와 저장되는 값의 범위가 다름

타입	메모리 크기		저장되는 값의 허용 범위	
byte	1byte*	8bit	$-2^7 \sim (2^7-1)$	-128 ~ 127
short	2byte	16bit	$-2^{15} \sim (2^{15}-1)$	-32,768 ~ 32,767
char	2byte	16bit	$0 \sim (2^{16}-1)$	0 ~ 65535 (유니코드)
int	4byte	32bit	$-2^{31} \sim (2^{31}-1)$	-2,147,483,648 ~ 2,147,483,647
long	8byte	64bit	$-2^{63} \sim (2^{63}-1)$	-9,223,372,036,854,775,808 ~ 9,223,372,036,854,775,807

* 1byte = 8bit, bit는 0과 1이 저장되는 단위

정수 타입 (정수 자료형)

- byte 타입 (1 byte)

```
byte var1 = -128;  
byte var2 = -30;  
byte var3 = 0;  
byte var4 = 30;  
byte var5 = 127;  
//byte var6 = 128; //결과일 예러(Type mismatch: cannot covert from int to byte)  
  
System.out.println(var1);  
System.out.println(var2);  
System.out.println(var3);  
System.out.println(var4);  
System.out.println(var5);
```

- 리터럴 (literal)
 - 프로그래머가 직접 입력한 값

정수 타입 (정수 자료형)

- long 타입 (8 byte)
 - 정수타입 리터럴은 자동으로 int형으로 인식됨
 - int 범위를 벗어나는 리터럴을 만들고 싶으면 맨 뒤에 L 혹은 l을 추가

```
long var1 = 10;  
long var2 = 20L;  
//long var3 = 10000000000000; //컴파일러는 int로 간주하기 때문에 에러 발생시킬  
long var4 = 10000000000000L;  
  
System.out.println(var1);  
System.out.println(var2);  
System.out.println(var4);
```


문자 타입 (문자 자료형)

- 문자 리터럴과 char
 - 문자 리터럴: 하나의 문자를 작은 따옴표(')로 감싼 것
 - 문자 리터럴을 유니코드로 저장할 수 있도록 char 타입 제공

```
char var1 = 'A';    //'A' 문자와 매핑되는 숫자: 65로 대입  
char var3 = '가';   //'가' 문자와 매핑되는 숫자: 44032로 대입
```

```
char c = 65;        //10진수 65와 매핑되는 문자: 'A'  
char c = 0x0041;    //16진수 0x0041과 매핑되는 문자: 'A'
```

실수 타입 (실수 자료형)

- float 과 double 타입
 - 실수타입 리터럴은 자동으로 double형으로 인식됨
 - float 범위를 벗어나는 리터럴을 만들고 싶으면 맨 뒤에 F 혹은 f를 추가

```
//정밀도 확인
float var1 = 0.1234567890123456789f;
double var2 = 0.1234567890123456789;
System.out.println("var1: " + var1);
System.out.println("var2: " + var2);
```

실수 타입 (실수 자료형)

- float 과 double 타입
 - e혹은 E가 포함된 리터럴: $5e2 = 5 \times 100$

논리 타입 (논리 자료형)

- boolean 타입
 - true 혹은 false로 구성됨

```
boolean stop = true;  
boolean stop = false;
```

- 주로 두 가지 상태값을 저장하는 경우에 사용

	연산식	
<code>int x = 10;</code>		
<code>boolean result =</code>	<code>(x == 20);</code>	//변수 x의 값이 20인가?
<code>boolean result =</code>	<code>(x != 20);</code>	//변수 x의 값이 20이 아닌가?
<code>boolean result =</code>	<code>(x > 20);</code>	//변수 x의 값이 20보다 큰가?
<code>boolean result =</code>	<code>(0 < x && x < 20);</code>	//변수 x의 값이 0보다 크고, 20보다 적은가?
<code>boolean result =</code>	<code>(x < 0 x > 200);</code>	//변수 x의 값이 0보다 적거나 200보다 큰가?

문자열 타입 (문자열 자료형)

- String 타입

- 문자열 (String): 큰 따옴표(“”)로 감싼 문자들

```
String var1 = "A";  
String var2 = "홍길동";
```

- 이스케이프 문자: 문자열 내부에 역슬래시(\)가 붙은 특수 문자

이스케이프 문자	
\"	" 문자 포함
\'	' 문자 포함
\\	\ 문자 포함
\u16진수	16진수 유니코드에 해당하는 문자 포함
\t	출력 시 탭만큼 띄움
\n	출력 시 줄바꿈(라인피드)
\r	출력 시 캐리지 리턴

문자열 타입 (문자열 자료형)

- String 타입
 - 참고: println 메소드는 print + \n 과 같음

```
String name = "홍길동";  
String job = "프로그래머";  
System.out.println(name);  
System.out.println(job);  
  
String str = "나는 \"자바\"를 배웁니다..";  
System.out.println(str);  
  
str = "번호\t이름\t직업 ";  
System.out.println(str);  
  
System.out.print("나는\n");  
System.out.print("자바를\n");  
System.out.print("배웁니다.");
```

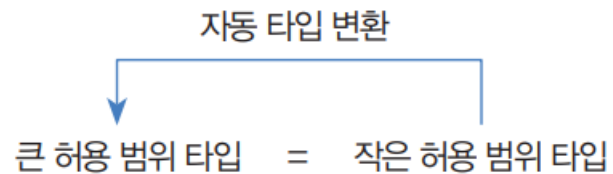


```
홍길동  
프로그래머  
나는 "자바"를 배웁니다..  
번호 이름 직업  
나는  
자바를  
배웁니다.
```

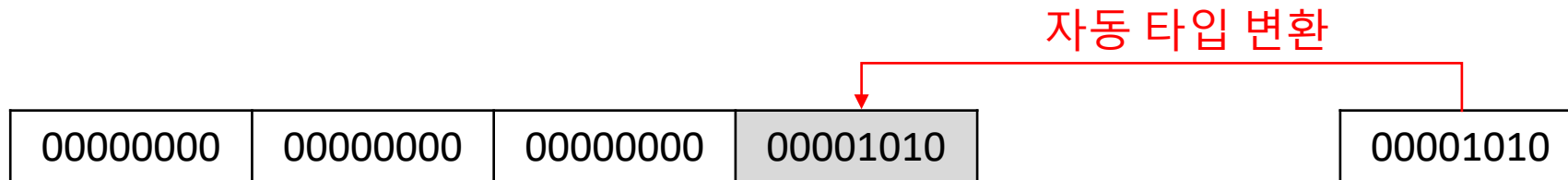
타입 변환 (형 변환)

- 자동 타입 변환 (자동 형 변환)

- 데이터 타입을 다른 타입으로 변환하는 것
- 값의 허용 범위가 작은 타입이 허용 범위가 큰 타입으로 대입될 때 발생



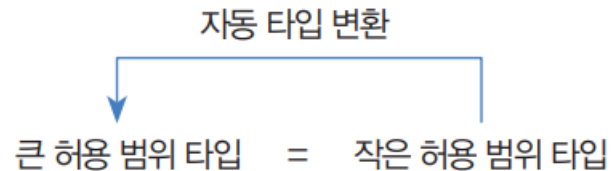
byte < short, char < int < long < float < double



타입 변환 (형 변환)

- 자동 타입 변환

- 정수 타입이 실수 타입으로 대입되면 무조건 자동 타입 변환이 됨
- 예외: char 타입보다 허용 범위가 작은 byte 타입은 char 타입으로 자동 변환될 수 없음



byte < short, char < int < long < float < double

타입 변환 (형 변환)

- 자동 타입 변환

```
// 자동 타입 변환
byte byteValue = 10;
int intValue = byteValue;
System.out.println("intValue: " + intValue);

char charValue = '가';
intValue = charValue;
System.out.println("가의 유니코드: " + intValue);

intValue = 50;
long longValue = intValue;
System.out.println("longValue: " + longValue);

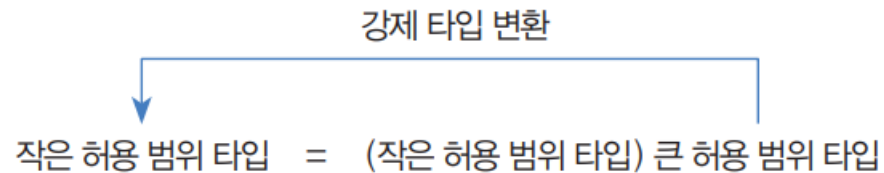
longValue = 100;
float floatValue = longValue;
System.out.println("floatValue: " + floatValue);

floatValue = 100.5F;
double doubleValue = floatValue;
System.out.println("doubleValue: " + doubleValue);
```

타입 변환 (형 변환)

- 강제 타입 변환 (강제 형 변환)

- 큰 허용 범위 타입을 작은 허용 범위 타입으로 쪼개어서 저장하는 것
- 캐스팅 연산자로 괄호()를 사용하며, 괄호 안에 들어가는 타입은 쪼개는 단위

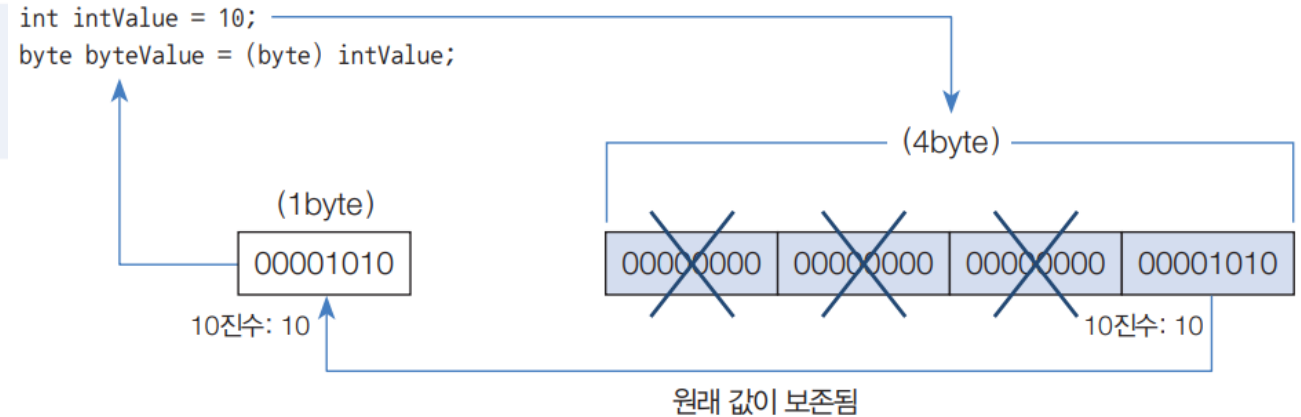


- long → int
- int → char
- 실수 → 정수

타입 변환 (형 변환)

- 강제 타입 변환
 - 예: `int` → `byte` 강제 타입 변환

```
int intValue = 10;  
byte byteValue = (byte) intValue; //강제 타입 변환
```



타입 변환 (형 변환)

- 강제 타입 변환

```
int var1 = 10;
byte var2 = (byte) var1;
System.out.println(var2);    //강제 타입 변환 후에 10이 그대로 유지

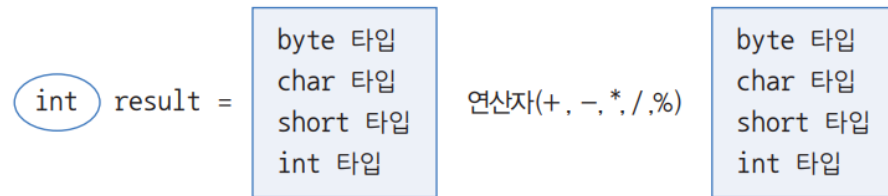
long var3 = 300;
int var4 = (int) var3;
System.out.println(var4);    //강제 타입 변환 후에 300이 그대로 유지

int var5 = 65;
char var6 = (char) var5;
System.out.println(var6);    //'A'가 출력

double var7 = 3.14;
int var8 = (int) var7;
System.out.println(var8);    //3이 출력
```

타입 변환 (형 변환)

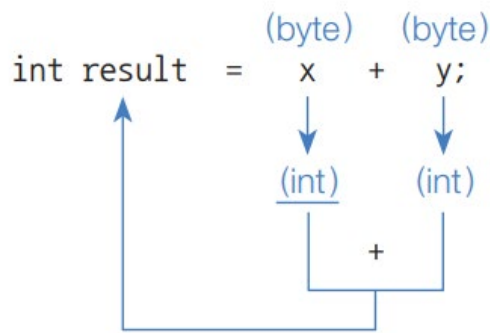
- 연산식에서 자동 타입 변환
 - 정수 타입 변수가 산술 연산식에서 피연산자로 사용되면 int 타입보다 작은 byte, short 타입 변수는 int 타입으로 자동 변환되어 연산 수행



byte 타입 변수가 피연산자로 사용된 경우	int 타입 변수가 피연산자로 사용된 경우
<pre>byte x = 10; byte y = 20; byte result = x + y; //컴파일 에러 int result = x + y;</pre>	<pre>int x = 10; int y = 20; int result = x + y;</pre>

타입 변환 (형 변환)

- 연산식에서 자동 타입 변환
 - 정수 타입 변수가 산술 연산식에서 피연산자로 사용되면 int 타입보다 작은 byte, short, char 타입 변수는 int 타입으로 자동 변환되어 연산 수행
 - byte 변수가 피연산자로 사용되면 변수값은 int 값으로 연산되며, 결과값 역시 byte 변수가 아닌 int 변수에 저장해야 함 (short, char 도 마찬가지)



```
char v6 = 'A';
char v7 = 1;
int result4 = v6 + v7; //int 타입으로 변환후 연산
System.out.println("result4: " + result4);
System.out.println("result4: " + (char)result4);
```

타입 변환 (형 변환)

- 연산식에서 자동 타입 변환
 - long 변수가 피연산자로 사용되면 변수값은 long 값으로 연산되며, 결과값 역시 long 변수에 저장해야 함

```
byte v3 = 10;  
int v4 = 100;  
long v5 = 1000L;  
long result3 = v3 + v4 + v5;    //long 타입으로 변환후 연산  
System.out.println("result3: " + result3);
```

타입 변환 (형 변환)

- 연산식에서 자동 타입 변환
 - double 변수가 피연산자로 사용되면 변수값은 double 값으로 연산되며, 결과값 역시 double 변수에 저장해야 함

```
int v9 = 10;  
double result6 = v9 / 4.0; //double 타입으로 변환후 연산  
System.out.println("result6: " + result6);
```

```
float v12 = 1;  
double v13 = 2;  
double result8 = v12 / v13; //double 타입이 연산에 포함되면 double 타입으로 변환후 연산  
System.out.println("result7: " + result8);
```


타입 변환 (형 변환)

- 문자열을 기본 타입으로 변환

변환 타입	사용 예
String → byte	<pre>String str = "10"; byte value = Byte.parseByte(str);</pre>
String → short	<pre>String str = "200"; short value = Short.parseShort(str);</pre>
String → int	<pre>String str = "3000000"; int value = Integer.parseInt(str);</pre>
String → long	<pre>String str = "4000000000000"; long value = Long.parseLong(str);</pre>
String → float	<pre>String str = "12.345"; float value = Float.parseFloat(str);</pre>
String → double	<pre>String str = "12.345"; double value = Double.parseDouble(str);</pre>
String → boolean	<pre>String str = "true"; boolean value = Boolean.parseBoolean(str);</pre>

타입 변환 (형 변환)

- 문자열을 기본 타입으로 변환
 - 기본 타입의 값을 문자열로 변경할 때는 `String.valueOf()` 메소드 사용

```
int value1 = Integer.parseInt(s: "10");
double value2 = Double.parseDouble(s: "3.14");
boolean value3 = Boolean.parseBoolean(s: "true");

System.out.println("value1: " + value1);
System.out.println("value2: " + value2);
System.out.println("value3: " + value3);

String str1 = String.valueOf(i: 10);
String str2 = String.valueOf(d: 3.14);
String str3 = String.valueOf(b: true);

System.out.println("str1: " + str1);
System.out.println("str2: " + str2);
System.out.println("str3: " + str3);
```

타입 변환 (형 변환)

- 문자열 결합 (+ 연산자)
 - 피연산자가 모두 숫자 → 덧셈 연산
 - 피연산자 중 문자열이 있을 때 → 문자열 자동 변환 후 결합

```
//숫자 연산
int result1 = 10 + 2 + 8;
System.out.println("result1: " + result1);

//결합 연산
String result2 = 10 + 2 + "8";
System.out.println("result2: " + result2);

String result3 = 10 + "2" + 8;
System.out.println("result3: " + result3);

String result4 = "10" + 2 + 8;
System.out.println("result4: " + result4);

String result5 = "10" + (2 + 8);
System.out.println("result5: " + result5);
```

변수값 출력

- printf() 메소드로 변수값 출력
 - printf()의 형식 문자열에서는 %와 conversation(변환 문자)를 필수로 작성하고 나머지는 생략 가능

```
int value = 123;
System.out.printf("상품의 가격:%d원\n", value);
System.out.printf("상품의 가격:%6d원\n", value);
System.out.printf("상품의 가격:%-6d원\n", value);
System.out.printf("상품의 가격:%06d원\n", value);

double area = 3.14159 * 10 * 10;
System.out.printf("반지름이 %d인 원의 넓이:%10.2f\n", 10, area);

String name = "홍길동";
String job = "도적";
System.out.printf("%6d | %-10s | %10s\n", 1, name, job);
```

변수값 출력

• 형식화된 문자열

형식화된 문자열		설명	출력 형태
정수	%d	정수	123
	%6d	정수 6자리, 왼쪽 공백	___123
	%-6d	정수 6자리, 오른쪽 공백	123___
	%06d	정수 6자리, 왼쪽 공백 0 채움	000123
실수	%f	정수 + 소수점 + 소수 6자리, 왼쪽 공백 0 채움	123.450000
	%10.2f	정수 7자리 + 소수점 + 소수 2자리, 왼쪽 공백	___123.45
	%-10.2f	정수 7자리 + 소수점 + 소수 2자리, 오른쪽 공백	123.45___
	%010.2f	정수 7자리 + 소수점 + 소수 2자리, 왼쪽 공백 0 채움	000123.45
문자열	%s	문자열	abc
	%6s	문자열 6자리, 왼쪽 빈자리 공백	___abc
	%-6s	문자열 6자리, 오른쪽 빈자리 공백	abc___

연산자

- 연산자
- 연산 방향과 우선순위

연산자

- 부호 연산자

- 부호 연산자는 변수의 부호를 유지하거나 변경

연산식		설명
+	피연산자	피연산자의 부호 유지
-	피연산자	피연산자의 부호 변경

- 증감 연산자

- 증감 연산자는 변수의 값을 1 증가시키거나 1 감소시킴

연산식		설명
++	피연산자	피연산자의 값을 1 증가시킴
--	피연산자	피연산자의 값을 1 감소시킴
피연산자	++	다른 연산을 수행한 후에 피연산자의 값을 1 증가시킴
피연산자	--	다른 연산을 수행한 후에 피연산자의 값을 1 감소시킴

연산자

- 부호 연산자

```
int x = -100;  
x = -x;  
System.out.println("x: " + x);  
  
byte b = 100;  
int y = -b;  
System.out.println("y: " + y);
```


연산자

- 증감 연산자

```
int x = 10;
int y = 10;
int z;

z = x++; // z = 10, x = 11
System.out.println("z=" + z);
System.out.println("x=" + x);

System.out.println("-----");
z = ++x; // z = 12, x = 12
System.out.println("z=" + z);
System.out.println("x=" + x);

System.out.println("-----");
z = ++x + y++; // x = 13, z = 23, y = 11
System.out.println("z=" + z);
System.out.println("x=" + x);
System.out.println("y=" + y);
```

연산자

- 증감 연산자

- 메소드 안에 증감 연산자가 사용되는 경우 순서에 유의

- `System.out.println(a++)` → `a`를 출력 후 `a`에 1 증가
- `System.out.println(++a)` → `a`에 1 증가 후 `a`를 출력
- `System.out.println(a++ + b++)` → `a+b`를 출력 후 `a`와 `b`에 1씩 증가
- `System.out.println(++a + ++b)` → `a`와 `b`에 1씩 증가 후 `a+b`를 출력

```
int a = 10;
int b = 10;

System.out.println(a++); // print 10, a = 11
System.out.println(++a); // print 12, a = 12
System.out.println(--b); // print 9, b = 9
System.out.println(b--); // print 9, b = 8
System.out.println(a++ + b++); // print 20, a = 13, b = 9
System.out.println(a-- + b++); // print 22, a = 12, b = 10
System.out.println(++a + ++b); // print 24, a = 13, b = 11
System.out.println(a++ + ++b); // print 25, a = 14, b = 12
System.out.println(++a + b--); // print 27, a = 15, b = 11
```

연산자

- 산술 연산자

- 더하기(+), 빼기(-), 곱하기(*), 나누기(/), 나머지(%)로 총 5개

연산식			설명
피연산자	+	피연산자	덧셈 연산
피연산자	-	피연산자	뺄셈 연산
피연산자	*	피연산자	곱셈 연산
피연산자	/	피연산자	나눗셈 연산
피연산자	%	피연산자	나눗셈의 나머지를 산출하는 연산

연산자

- 산술 연산자

- 더하기(+), 빼기(-), 곱하기(*), 나누기(/), 나머지(%)로 총 5개

```
byte v1 = 10;
byte v2 = 4;
int v3 = 5;
long v4 = 10L;

int result1 = v1 + v2;      // 모든 피연산자는 int 타입으로 자동 변환 후 연산
System.out.println("result1: " + result1);

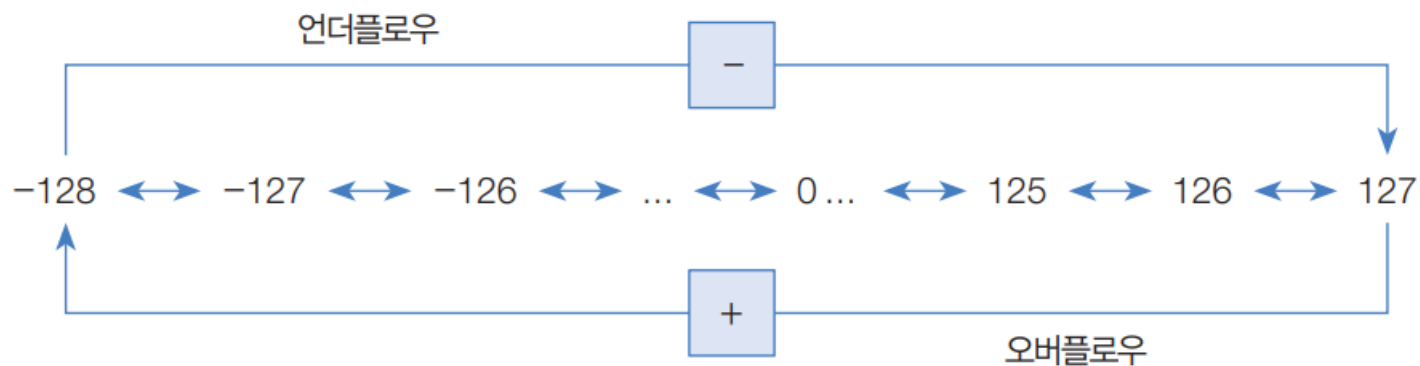
long result2 = v1 + v2 - v4; // 모든 피연산자는 long 타입으로 자동 변환 후 연산
System.out.println("result2: " + result2);

double result3 = (double) v1 / v2; // double 타입으로 강제 변환 후 연산
System.out.println("result3: " + result3);

int result4 = v1 % v2;
System.out.println("result4: " + result4);
```

연산자

- 오버플로우
 - 변수 타입이 허용하는 최대값을 초과하는 것
- 언더플로우
 - 변수 타입이 허용하는 최소값을 초과하는 것



0	1	1	1	1	1	1	1	→	127
0	1	1	1	1	1	1	0	→	126
0	0	0	0	0	0	0	1	→	1
0	0	0	0	0	0	0	0	→	0
1	1	1	1	1	1	1	1	→	-1
1	0	0	0	0	0	0	1	→	-127
1	0	0	0	0	0	0	0	→	-128

연산자

- 오버플로우
 - 변수 타입이 허용하는 최대값을 초과하는 것
- 언더플로우
 - 변수 타입이 허용하는 최소값을 초과하는 것

```
byte b1 = 127;  
b1++;  
System.out.println(b1);  
  
byte b2 = -128;  
b2--;  
System.out.println(b2);
```

정수 연산

- 정확한 계산은 정수 연산으로
 - 산술 연산을 정확하게 계산하려면 실수 타입을 사용하지 않는 것이 좋음

```
int apple = 1;  
double pieceUnit = 0.1;  
int number = 7;  
  
double result = apple - number*pieceUnit;  
System.out.println("사과 1개에서 남은 양: " + result);
```

```
사과 1개에서 남은 양: 0.29999999999999993
```

정수 연산

- 정확한 계산은 정수 연산으로
 - 산술 연산을 정확하게 계산하려면 실수 타입을 사용하지 않는 것이 좋음

```
int apple = 1;
int totalPieces = apple * 10;
int number = 7;

int result = totalPieces - number;
System.out.println("10조각에서 남은 조각: " + result);
System.out.println("사과 1개에서 남은 양: " + result/10.0);
```

```
10조각에서 남은 조각: 3
사과 1개에서 남은 양: 0.3
```


나눗셈 연산

- 나눗셈 또는 나머지 연산에서 예외 방지
 - 좌측 피연산자가 정수이고 우측 피연산자가 0일 경우 런타임 에러 발생
 - 좌측 피연산자가 실수이고 우측 피연산자가 0.0 또는 0.0f일 경우 연산의 결과가 Infinity 혹은 NaN이 됨

```
5 / 0.0 → Infinity
```

```
5 % 0.0 → NaN
```

- Infinity 또는 NaN 상태에서 연산을 수행하면 데이터가 망가짐
- Double.isInfinite()와 Double.isNaN을 사용해 연산의 결과가 Infinity또는 NaN인지 확인하는 것이 필요

나눗셈 연산

- NaN과 Infinity 처리

```
int x = 5;
double y = 0.0;
double z = x / y;
//double z = x % y;

//잘못된 코드
System.out.println(z + 2);

//알맞은 코드
if(Double.isInfinite(z) || Double.isNaN(z)) {
    System.out.println("값 산출 불가");
} else {
    System.out.println(z + 2);
}
```

연산자

- 비교 연산자

- 피연산자들의 값을 비교하여 boolean 타입인 true/false를 반환

구분	연산식			설명
동등 비교	피연산자1	==	피연산자2	두 피연산자의 값이 같은지를 검사
	피연산자1	!=	피연산자2	두 피연산자의 값이 다른지를 검사
크기 비교	피연산자1	>	피연산자2	피연산자1이 큰지를 검사
	피연산자1	>=	피연산자2	피연산자1이 크거나 같은지를 검사
	피연산자1	<	피연산자2	피연산자1이 작은지를 검사
	피연산자1	<=	피연산자2	피연산자1이 작거나 같은지를 검사

- 문자열을 비교할 때는 ==, != 연산자 대신 equals()를 사용

연산자

- 비교 연산자

```
int num1 = 10;
int num2 = 10;
boolean result1 = (num1 == num2);
boolean result2 = (num1 != num2);
boolean result3 = (num1 <= num2);
System.out.println("result1: " + result1);
System.out.println("result2: " + result2);
System.out.println("result3: " + result3);

char char1 = 'A';
char char2 = 'B';
boolean result4 = (char1 < char2); // 65 < 66
System.out.println("result4: " + result4);

int num3 = 1;
double num4 = 1.0;
boolean result5 = (num3 == num4);
System.out.println("result5: " + result5);

float num5 = 0.1f;
double num6 = 0.1;
boolean result6 = (num5 == num6);
boolean result7 = (num5 == (float)num6);
System.out.println("result6: " + result6);
System.out.println("result7: " + result7);
```

연산자

- 비교 연산자

- 문자열을 비교할 때는 ==, != 연산자 대신 equals()를 사용
- ==, != 연산자는 주소값을 비교 → 이후에 자세히 설명
- equals()는 실제값을 비교

```
String str1 = "자바";
String str2 = "Java";
String str3 = "Java";
boolean result8 = (str1.equals(str2));
boolean result9 = (! str1.equals(str2));
boolean result10 = (str1 == str3);
boolean result11 = (str2.equals(str3));
System.out.println("result8: " + result8);
System.out.println("result9: " + result9);
System.out.println("result10: " + result10); // false
System.out.println("result11: " + result11); // true
```

연산자

- 논리 연산자

- 논리곱(&&), 논리합(||), 배타적 논리합(^) 그리고 논리 부정(!) 연산

구분	연산식			결과	설명
AND (논리곱)	true	&& 또는 &	true	true	피연산자 모두가 true일 경우에만 연산 결과가 true
	true		false	false	
	false		true	false	
	false		false	false	
OR (논리합)	true	 또는 	true	true	피연산자 중 하나만 true이면 연산 결과는 true
	true		false	true	
	false		true	true	
	false		false	false	
XOR (배타적 논리합)	true	^	true	false	피연산자가 하나는 true이고 다른 하나가 false일 경우에만 연산 결과가 true
	true		false	true	
	false		true	true	
	false		false	false	
NOT (논리 부정)		!	true	false	피연산자의 논리값을 바꿈
			false	true	

연산자

- 논리 연산자

- 논리곱(&&), 논리합(||), 배타적 논리합(^) 그리고 논리 부정(!) 연산
 - &&는 앞의 피연산자가 false라면 뒤의 피연산자는 계산하지 않고 false 반환
 - ||는 앞의 피연산자가 true라면 뒤의 피연산자는 계산하지 않고 true 반환

```
int charCode = 'A';
//int charCode = 'a';
//int charCode = '5';

if( (65<=charCode) & (charCode<=90) ) {
    System.out.println("대문자 이군요");
}

if( (97<=charCode) && (charCode<=122) ) {
    System.out.println("소문자 이군요");
}

if( (48<=charCode) && (charCode<=57) ) {
    System.out.println("0~9 숫자 이군요");
}
```

```
int value = 6;
//int value = 7;

if( (value%2==0) | (value%3==0) ) {
    System.out.println("2 또는 3의 배수 이군요");
}

boolean result = (value%2==0) || (value%3==0);
if( !result ) {
    System.out.println("2 또는 3의 배수가 아니군요.");
}
```

연산자

- 비트 논리 연산자 (byte, short, int, long)
 - bit 단위로 논리 연산을 수행, 0과 1이 피연산자가 됨

구분	연산식			결과	설명
AND (논리곱)	1	&	1	1	두 비트 모두 1일 경우에만 연산 결과가 1
	1		0	0	
	0		1	0	
	0		0	0	
OR (논리합)	1		1	1	두 비트 중 하나만 1이면 연산 결과는 1
	1		0	1	
	0		1	1	
	0		0	0	
XOR (배타적 논리합)	1	^	1	0	두 비트 중 하나는 1이고 다른 하나가 0일 경우 연산 결과는 1
	1		0	1	
	0		1	1	
	0		0	0	
NOT (논리 부정)		~	1	0	보수
			0	1	

연산자

- 비트 논리 연산자 (byte, short, int, long)
 - bit 단위로 논리 연산을 수행, 0과 1이 피연산자가 됨

```
System.out.println("45 & 25 = " + (45 & 25));  
System.out.println("45 | 25 = " + (45 | 25));  
System.out.println("45 ^ 25 = " + (45 ^ 25));  
System.out.println("~45 = " + (~45));
```

• $45 = 32 + 8 + 4 + 1 \rightarrow$

0	0	1	0	1	1	0	1
---	---	---	---	---	---	---	---

• $25 = 16 + 8 + 1 \rightarrow$

0	0	0	1	1	0	0	1
---	---	---	---	---	---	---	---

• $45 \& 25 \rightarrow$

0	0	0	0	1	0	0	1
---	---	---	---	---	---	---	---

= 9

• $45 | 25 \rightarrow$

0	0	1	1	1	1	0	1
---	---	---	---	---	---	---	---

= 61

• $45 \wedge 25 \rightarrow$

0	0	1	1	0	1	0	0
---	---	---	---	---	---	---	---

= 52

• $\sim 45 \rightarrow$

1	1	0	1	0	0	1	0
---	---	---	---	---	---	---	---

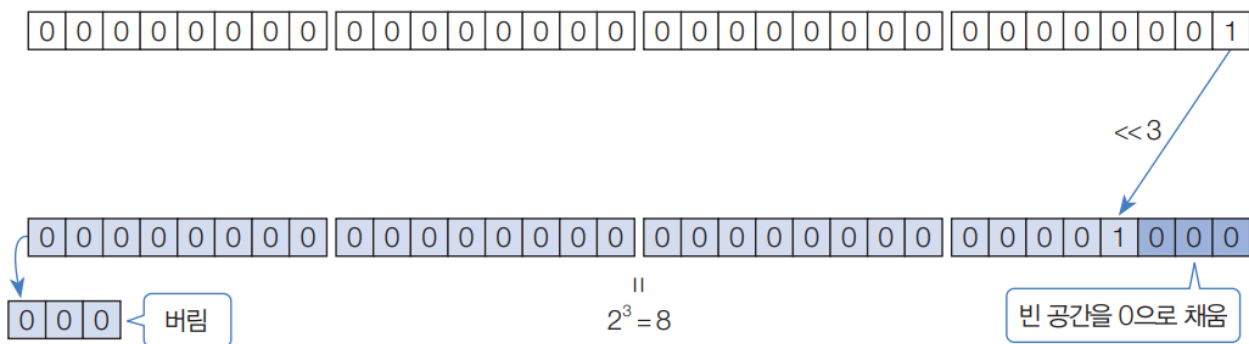
= -46

연산자

• 비트 이동 연산자

- 비트를 좌측 또는 우측으로 밀어서 이동 시키는 연산

구분	연산식			설명
이동 (shift)	a	《	b	정수 a의 각 비트를 b만큼 왼쪽으로 이동 오른쪽 빈자리는 0으로 채움 $a \times 2^b$ 와 동일한 결과가 됨
	a	》	b	정수 a의 각 비트를 b만큼 오른쪽으로 이동 왼쪽 빈자리는 최상위 부호 비트와 같은 값으로 채움 $a / 2^b$ 와 동일한 결과가 됨
	a	》》	b	정수 a의 각 비트를 b만큼 오른쪽으로 이동 왼쪽 빈자리는 0으로 채움



연산자

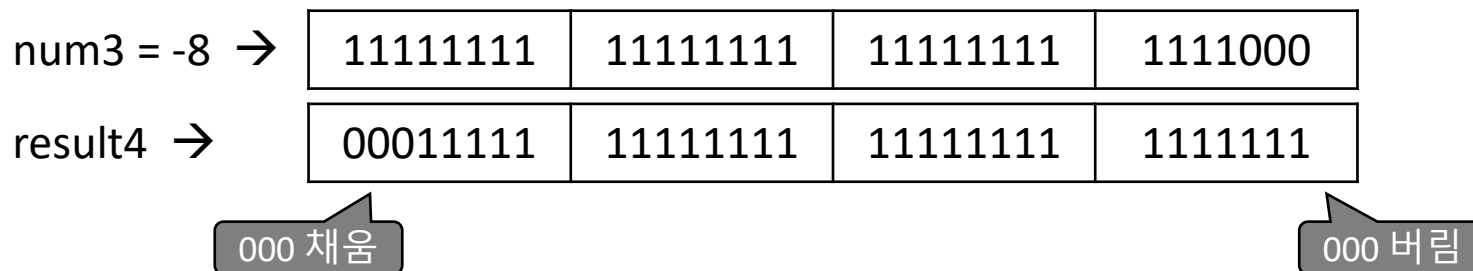
- 비트 이동 연산자

- 비트를 좌측 또는 우측으로 밀어서 이동 시키는 연산

```
int num1 = 1;
int result1 = num1 << 3;
System.out.println("result1: " + result1);

int num2 = -8;
int result3 = num2 >> 3;
System.out.println("result2: " + result3);

int num3 = -8;
int result4 = num3 >>> 3;
System.out.println("result3: " + result4);
```



연산자

• 대입 연산자

- 우측 피연산자의 값을 좌측 피연산자인 변수에 대입

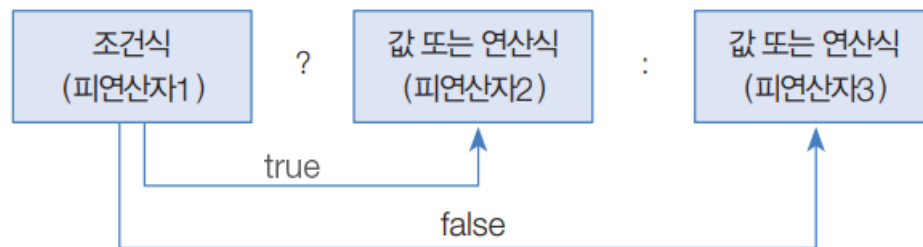
```
int result = 0;
result += 10;
System.out.println("result=" + result);
result -= 5;
System.out.println("result=" + result);
result *= 3;
System.out.println("result=" + result);
result /= 5;
System.out.println("result=" + result);
result %= 3;
System.out.println("result=" + result);
```

구분	연산식			설명
단순 대입 연산자	변수	=	피연산자	우측의 피연산자의 값을 변수에 저장
	변수	+=	피연산자	우측의 피연산자의 값을 변수의 값과 더한 후에 다시 변수에 저장 (변수 = 변수 + 피연산자)
복합 대입 연산자	변수	-=	피연산자	우측의 피연산자의 값을 변수의 값에서 뺀 후에 다시 변수에 저장 (변수 = 변수 - 피연산자)
	변수	*=	피연산자	우측의 피연산자의 값을 변수의 값과 곱한 후에 다시 변수에 저장 (변수 = 변수 * 피연산자)
	변수	/=	피연산자	우측의 피연산자의 값으로 변수의 값을 나눈 후에 다시 변수에 저장 (변수 = 변수 / 피연산자)
	변수	%=	피연산자	우측의 피연산자의 값으로 변수의 값을 나눈 후에 나머지를 변수에 저장 (변수 = 변수 % 피연산자)
	변수	&=	피연산자	우측의 피연산자의 값과 변수의 값을 & 연산 후 결과를 변수에 저장 (변수 = 변수 & 피연산자)
	변수	=	피연산자	우측의 피연산자의 값과 변수의 값을 연산 후 결과를 변수에 저장 (변수 = 변수 피연산자)
	변수	^=	피연산자	우측의 피연산자의 값과 변수의 값을 ^ 연산 후 결과를 변수에 저장 (변수 = 변수 ^ 피연산자)
	변수	<<=	피연산자	우측의 피연산자의 값과 변수의 값을 << 연산 후 결과를 변수에 저장 (변수 = 변수 << 피연산자)
	변수	>>=	피연산자	우측의 피연산자의 값과 변수의 값을 >> 연산 후 결과를 변수에 저장 (변수 = 변수 >> 피연산자)
	변수	>>>=	피연산자	우측의 피연산자의 값과 변수의 값을 >>> 연산 후 결과를 변수에 저장 (변수 = 변수 >>> 피연산자)

연산자

- 삼항 연산자

- 총 3개의 피연산자를 가짐
- ? 앞의 피연산자는 boolean 변수 또는 조건식. 이 값이 true이면 (:) 앞의 피연산자가 선택되고 false이면 뒤의 피연산자가 선택됨



```
int score = 85;  
char grade = (score > 90) ? 'A' : ( (score > 80) ? 'B' : 'C' );  
System.out.println(score + "점은 " + grade + "등급입니다.");
```

연산의 방향과 우선순위

• 연산 수행 순서

연산자	연산 방향	우선순위
증감(++ , --), 부호(+, -), 비트(~), 논리(!)	←	높음 ↕ 낮음
산술(*, /, %)	→	
산술(+, -)	→	
쉬프트(<<, >>, >>>)	→	
비교(<, >, <=, >=, instanceof)	→	
비교(==, !=)	→	
논리(&)	→	
논리(^)	→	
논리()	→	
논리(&&)	→	
논리()	→	
조건(?:)	→	
대입(=, +=, -=, *=, /=, %=, &=, ^=, =, <<=, >>=, >>>=)	←	

